

บทที่ 5

บทวิจารณ์และสรุป

5.1 บทสรุป

ในปัจจุบันการสร้างเกมนั้นต้องทำการเขียนโค้ด ติดต่อกับ Hardware หลายส่วน รวมไปถึงการเขียนโค้ดเกี่ยวกับ การนำเข้า โมเดล และการจำลองการเคลื่อนที่ต่างๆ ทำให้กว่าที่ผู้พัฒนาจะสร้างเกมขึ้นมาได้นั้นใช้เวลานาน

ดังนั้น เกมเอนจินที่พัฒนาขึ้นสามารถช่วยให้ผู้พัฒนาเกมสร้างเกมได้ง่าย และทำให้โค้ดของเกมนั้นสั้นลง ง่ายต่อการตรวจสอบและแก้ไข โดยภายในเกมเอนจินจะประกอบไปด้วย Tool ต่างๆ ให้ผู้พัฒนานำไปใช้ ซึ่งผู้พัฒนาไม่จำเป็นต้องเขียนโค้ดเองทั้งหมด แต่เกมเอนจินก็มีความยืดหยุ่นที่อนุญาตให้ผู้พัฒนาโปรแกรมนั้น เขียนส่วนเพิ่มเติมที่นอกเหนือจากที่มีในเกมเอนจินได้

5.2 วิเคราะห์สิ่งที่ได้จากโครงงาน

จากที่ทำการพัฒนาเอนจินมา พบว่าเอนจินนั้นสามารถทำงานได้ในแบบพื้นฐานทั่วไปและยังมีบางส่วนที่ยังไม่สมบูรณ์ เนื่องจากในการพัฒนาเกมเอนจินนั้นถ้าต้องการให้ออกมาสมบูรณ์แบบ ควรจะทำการศึกษาและพัฒนาเฉพาะส่วนที่มีอยู่ในเกมเอนจิน เพื่อให้ได้ผลลัพธ์ในส่วนนั้นดีที่สุด ซึ่งถ้าแบ่งกันพัฒนาแต่ละส่วนแล้วนำมารวมกันก็จะได้เกมเอนจินที่ออกมาสมบูรณ์แบบในทุกด้าน อย่างไรก็ตามโครงงานที่ได้ถือว่าประสบความสำเร็จในระดับหนึ่งโดยสามารถนำไปสร้างตัวอย่างเกมเล็กๆ ได้ถึงแม้จะไม่สะดวกนักก็ตาม เนื่องจากชิ้นงานยังไม่ได้มีการปรับแต่งให้ง่ายต่อการใช้

5.3 ปัญหาอุปสรรคและแนวทางแก้ไข

- 5.3.1 การใช้ Scene Graph แม้จะช่วยจัดการเรื่องของ Transformation ได้ดี แต่การแสดงผลในแต่ละโหนดนั้นได้เป็นไปตามลำดับล่วงหน้าแล้ว (Depth First Traverse) ถึงแม้ว่า Depth Buffer จะช่วยแก้ปัญหาใน Depth Sorting ได้แต่ในบางกรณีมีความจำเป็นที่ต้องแสดงผลตามลำดับก่อนหลัง เช่น การแสดงภาพวัตถุ โปร่งแสงเป็นต้นทำให้ไม่สะดวกต่อผู้ใช้ แนวทางแก้ไขคือ มีอัลกอริทึม ที่ช่วยในการจัดเรียง ความลึกของวัตถุมาช่วยจัดการลำดับของโหนด ใน Scene Graph เช่น BSP Tree เป็นต้น

- 5.3.2. นอกจากข้อแรก ลำดับในการวาดของ Scene Graph ที่กำหนดให้สร้าง Vertex และ index Buffer สำหรับแต่ละวัตถุแล้วทำการวาด พบได้ว่าในบาง Graphic API สามารถที่จะ Render วัตถุ หรือ Primitive ในปริมาณมากๆ ต่อการวาด หนึ่งครั้งได้ดีกว่าการแสดงผลทีละน้อย และ ยังสูญเสียเวลาในการเปลี่ยนแปลง Render State แนวทางการแก้ปัญหา มีการสะสมวัตถุที่มีคุณสมบัติคล้ายกันเช่น ใช้ Texture เดียวกัน หรือใช้ Render State เหมือนกันรวมเข้าไว้ในการวาดเพียงครั้งเดียว
- 5.3.3. ในการพัฒนาเครื่องที่ใช้ร่วมกับโปรแกรมสามมิติ อื่นๆ เช่นการพัฒนาเครื่องมือที่ใช้การ Export file จาก 3Ds Studio Max มีความยากลำบากในเรื่องของระบบพิกัดแกนไม่เหมือนกันเป็นผลให้การพัฒนาเป็นไปโดยไม่สมบูรณ์ แนวทางแก้ปัญหาเพิ่มเติมส่วนที่ช่วยในการแปลงระบบพิกัดขึ้น
- 5.3.4. การทำ Global Effect หรือ Effect ที่มีผลมาจากการแสดงวัตถุอื่นหรือมีผลกับการแสดงวัตถุอื่น สามารถทำได้ไม่สะดวกนัก ไม่มีทางแก้ไขให้สะดวกขึ้นได้ เนื่องจาก Scene Graph มีลำดับการวาด นอกเสียจากจะต้องสร้าง เมธอด ที่ใช้ในการวาดและจัดการผลที่เกิดขึ้นเหล่านั้นด้วยตนเองซึ่งผู้พัฒนาจะต้องมีความรู้เกี่ยวกับระบบของเอนจินได้เป็นอย่างดี
- 5.3.5. ในการกำหนด Force และ Torque ให้กับ Rigid Body ตั้งแต่สองชิ้นขึ้นไป ถ้าต้องการให้ Force และ Torque แตกต่างกันในการออกแบบกำหนดให้ผู้ใช้สามารถใส่ฟังก์ชันในการคำนวณ Force และ Torque เองได้เพื่อความยืดหยุ่น แต่ก็ส่งผลให้ ต้องทำการสร้างฟังก์ชันสำหรับ Force และ Torque ขึ้นมาใหม่แล้วกำหนดให้กับ Rigid Body ที่เราต้องการ ทำให้ไม่สะดวกนัก แนวทางการแก้ไข ทำการสร้าง State ขึ้นมาว่าในแต่ละ State ต้องการให้ Rigid Body ขึ้นนั้นมี Force และ Torque เป็นอย่างไร โดย State นี้จะถูกเก็บอยู่ใน ฟังก์ชัน Force และ Torque ด้วย ทำให้เราสามารถมีฟังก์ชัน Force และ Torque เพียงฟังก์ชันเดียวในการกำหนดแรงให้กับวัตถุหลายชิ้น
- 5.3.6. ในการคำนวณ Inertia รูปทรงพื้นฐานต่างๆ ควรจะมีฟังก์ชันในการคำนวณอย่างง่าย เช่น วงกลม กล่อง สามเหลี่ยม
- 5.3.7. Input Register ของ Vertex Shader มีความหลากหลายมากกว่า Vertex Data Format ที่ใช้ในปัจจุบันส่งผลให้ใช้ความสามารถของ Shader ได้ไม่เต็มที่ เนื่องจากการออกแบบ

ส่วนที่สำหรับสนับสนุน Shader นั้น กระทำขึ้นหลังจากส่วนของ Scene Graph ที่ดำเนินงานได้อย่างสมบูรณ์แล้วจึงพยายามที่จะคงรูปแบบเดิมไว้ แนวทางแก้ไขใช้รูปแบบการกำหนด Vertex Format ใหม่สำหรับการแสดงผลที่ใช้ Shader (ปัญหาที่เกิดขึ้นใน DirectX)

5.4 แนวทางพัฒนาต่อ

- 5.4.1. ทำการปรับแต่ง เอนจินให้ใช้งานได้ง่ายขึ้น รวมไปถึงแก้ปัญหาต่างๆ ในส่วนที่กล่าวมาแล้ว
- 5.4.2. เพิ่มเทคนิคพิเศษต่างๆ ที่เป็นที่ยอมรับ เนื่องจากผู้พัฒนามีเวลาไม่มากพอในการพัฒนา
- 5.4.3. เทคนิค CLOD (Continuous Level Of Detail) ที่ใช้อยู่ในตอนนี้เป็นเทคนิคที่ค่อนข้างล้าหลังเนื่องจาก Graphic Card ในปัจจุบันมีความสามารถในการแสดงผลมากขึ้น อีกทั้งยังมี Pixel และ Vertex Process ที่ดียิ่งขึ้น จึงควรที่จะปรับปรุงวิธีการใหม่และนำไปประมวลผลบน Graphic Card แทนที่จะคำนวณบน CPU
- 5.4.4. ในการพัฒนาเกมส์จริงจริงนั้นคือการทำให้ interpolation ในรูปแบบต่างๆ จึงสมควรที่จะมีคลาสที่สนับสนุนเรื่องเหล่านี้ไว้ด้วย
- 5.4.5. ขยายการคำนวณเกี่ยวกับ Skin ใน Skeletal animation ไปบน GPU
- 5.4.6. ในการพัฒนาเกมส์ การออกแบบจากมีความสำคัญจึงควรพัฒนาเครื่องมือในการออกแบบฉากและพัฒนาในส่วนของ Scene Graph ในสามารภ บันทึกรหัส หรือ โหลด ฉากในปัจจุบันได้
- 5.4.7. เพิ่มการคำนวณ ฟิสิกส์ที่ทันสมัยอื่นๆ เช่น แรงลอยตัว ของวัตถุในของเหลว ข้อต่อรูปแบบต่างๆ รวมถึง ฟิสิกส์สำหรับ ยานพาหนะ